

Báo cáo

Nâng cao 1. Reverse Engineering với x64dbg/IDA

Mẫu 2

MỤC LỤC

1. Phân tích tĩnh với IDA.....	3
1.1 Thông tin về mẫu.....	3
1.2 Nhận định.....	5
2. Phân tích động.....	6
3. Kết luận.....	8

1. Phân tích tĩnh với IDA

1.1 Thông tin về mẫu

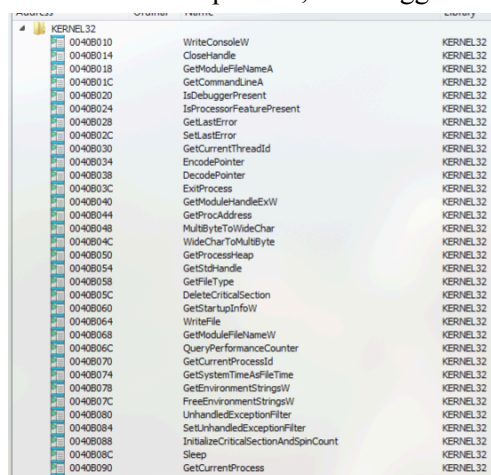
a. Tên file: RegistryRunKey.exe.

b. Kiểu file: PE32 executable for MS Windows 6.00 (console), Intel i386, 5 sections.

c. Các API đáng ngờ

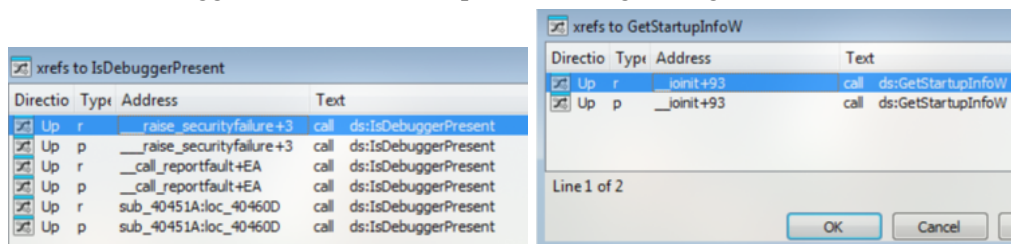
Dựa vào mục Imports của IDA, có thể thấy chương trình đã nạp các API từ 3 thư viện kernel32.dll, user32.dll, advapi32.dll. Các API đáng ngờ:

- Bộ 3 API từ advapi32.dll: RegCloseKey, RegOpenKeyA, RegSetKeyValueA.
- GetStartupInfoW, IsDebuggerPresent, GetModuleFileName, CreateFileW, WriteFile.



Address	API Name	Module
0040B010	WriteConsoleW	KERNEL32
0040B014	CloseHandle	KERNEL32
0040B018	GetModuleFileNameA	KERNEL32
0040B01C	GetCommandLineA	KERNEL32
0040B020	IsDebuggerPresent	KERNEL32
0040B024	IsProcessorFeaturePresent	KERNEL32
0040B028	GetLastError	KERNEL32
0040B02C	SetLastError	KERNEL32
0040B030	GetCurrentThreadId	KERNEL32
0040B034	EncodePointer	KERNEL32
0040B038	DecodePointer	KERNEL32
0040B03C	ExitProcess	KERNEL32
0040B040	GetModuleHandleExW	KERNEL32
0040B044	GetProcAddress	KERNEL32
0040B048	MultByteToWideChar	KERNEL32
0040B04C	WideCharToMultiByte	KERNEL32
0040B050	GetProcessHeap	KERNEL32
0040B054	GetStdHandle	KERNEL32
0040B058	GetFileType	KERNEL32
0040B05C	DeleteCriticalSection	KERNEL32
0040B060	GetStartupInfoW	KERNEL32
0040B064	WriteFile	KERNEL32
0040B068	GetModuleFileNameW	KERNEL32
0040B06C	QueryPerformanceCounter	KERNEL32
0040B070	GetCurrentProcessId	KERNEL32
0040B074	GetSystemTimeAsFileTime	KERNEL32
0040B078	GetEnvironmentStringsW	KERNEL32
0040B07C	FreeEnvironmentStringsW	KERNEL32
0040B080	UnhandledExceptionFilter	KERNEL32
0040B084	SetUnhandledExceptionFilter	KERNEL32
0040B088	InitializeCriticalSectionAndSpinCount	KERNEL32
0040B08C	Sleep	KERNEL32
0040B090	GetCurrentProcess	KERNEL32

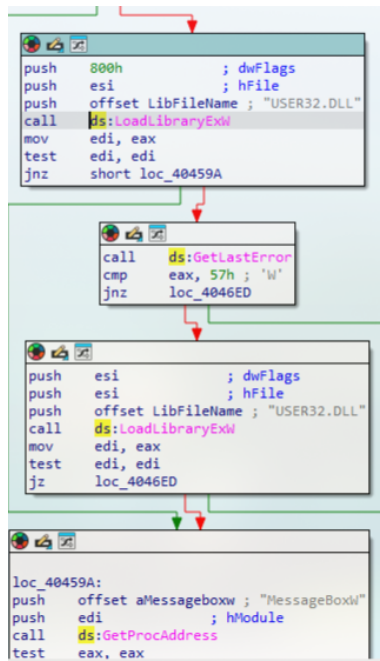
Các API IsDebuggerPresent, GetStartupInfoW không được gọi từ main nên có thể bỏ qua.



Direction	Type	Address	Text
Up	r	raise_securityfailure+3	call ds:IsDebuggerPresent
Up	p	__raise_securityfailure+3	call ds:IsDebuggerPresent
Up	r	__call_reportfault+EA	call ds:IsDebuggerPresent
Up	p	__call_reportfault+EA	call ds:IsDebuggerPresent
Up	r	sub_40451A:loc_40460D	call ds:IsDebuggerPresent
Up	p	sub_40451A:loc_40460D	call ds:IsDebuggerPresent

Direction	Type	Address	Text
Up	r	__init+93	call ds:GetStartupInfoW
Up	p	__init+93	call ds:GetStartupInfoW

Ngoài ra có LoadLibraryExW và GetProcAddress thường được dùng để nạp thêm API từ nguồn khác ([tham khảo](#)). Ở đây, Chúng được dùng để gọi MessageBoxW.



d. Strings đáng ngờ (tính năng View -> Open subviews -> Strings)

"SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run";

"DONE Registry Run Key ><";

"C:\\Users\\Admin\\Documents\\Visual Studio

2013\\Projects\\Persistence\\RegistryRunKey\\Release\\RegistryRunKey.pdb";

79	.rdata:0040...	00000007	C	1#QNAN
79	.rdata:0040...	0000002E	C	SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run
79	.rdata:0040...	0000000F	C	RegistryRunKey
79	.rdata:0040...	00000019	C	DONE Registry Run Key ><
79	.rdata:0040...	0000006B	C - U...	C:\\Users\\Admin\\Documents\\Visual Studio 2013\\Projects\\Persistence\\RegistryRunKey\\Release\\RegistryRunKey.pdb
79	.rdata:0040...	00000013	C	GetModuleFileNameA
79	.rdata:0040...	0000000D	C	KERNEL32.dll
79	.rdata:0040...	0000000C	C	MessageBoxA

e. Dịch ngược

Mã giả của hàm main:

```

1 int __cdecl main(int argc, const char **argv, const char **envp)
2 {
3     HKEY phkResult; // [esp+0h] [ebp-10Ch] BYREF
4     CHAR Filename[260]; // [esp+4h] [ebp-108h] BYREF
5
6     GetModuleFileNameA(0, Filename, 0x104u);
7     RegOpenKeyA(HKEY_CURRENT_USER, "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run", &phkResult);
8     RegSetKeyValueA(phkResult, 0, "RegistryRunKey", 1u, Filename, 0x104u);
9     RegCloseKey(phkResult);
10    MessageBoxA(0, "DONE Registry Run Key ><", "RegistryRunKey", 0);
11    return 0;
12 }

```

Các biến:

- phkResult: handle đến 1 key trong Registry.
- Filename: từ dòng 6, Filename chứa path của file chương trình hiện tại.

Dòng 7: Mở key Run trong HKCU, trả handle về phkResult.

Dòng 8: Đặt giá trị từ Filename, tức path của chương trình hiện tại, làm 1 giá trị trong key Run.

1.2 Nhận định

Chương trình thực hiện gán path của chính nó vào key Run của Registry, dẫn đến persistence.

2. Phân tích động

Vì chương trình đã cho thuộc kiến trúc 32-bit, sử dụng x32dbg.

Thực hiện đặt breakpoint tại các API xuất hiện trong mã giả.

Type	Address	Module/Label/Exception	State	Disassembly
Software	7646CB05	<advapi32.dll.RegOpenKeyA>	Disabled	mov edi,edi
	764C1F6F	<advapi32.dll.RegSetValueA>	Disabled	mov edi,edi
	76611491	<kernel32.dll.GetModuleFileNameA>	Enabled	mov edi,edi
	76613062	<kernel32.dll.RegCloseKey>	Disabled	push 10

Đầu tiên là API GetModuleFileNameA. Cho chương trình chạy đến breakpoint này.

76611491	8BFF	mov edi,edi	GetModuleFileNameA
76611493	55	push ebp	
76611494	8BEC	mov ebp,esp	
76611496	5D	pop ebp	
76611497	EB 05	jmp <JMP.&GetModuleFileNameA>	

Quan sát stack hiện tại và giá trị của tham số thứ 2 (Filename, địa chỉ 012F1520). Hiện tại, biến này chưa lưu giá trị gì.

Default (stdcall)	
1:	[esp+4] 00000000
2:	[esp+8] 012F1520 registryrunkey.012F1520
3:	[esp+C] 00000104
4:	[esp+10] 00000000
5:	[esp+14] 00000000

001BF894	012E1E01	return to registryrunkey.012E1E01 from ???
001BF898	00000000	
001BF89C	012F1520	registryrunkey.012F1520
001BF8A0	00000104	
001BF8A4	00000000	
001BF8A8	00000000	
001BF8AC	00000000	

Address	Hex	ASCII
012F1520	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1530	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1540	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1550	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1560	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1570	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1580	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1590	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F15A0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F15B0	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Cho chương trình Execute till return và quan sát lại. Lúc này, Filename đã lưu giữ path của chương trình hiện tại.

Address	Hex	ASCII
012F1520	43 3A 5C 55 73 65 72 73 5C 74 72 61 6E 67 5C 44	C:\users\trang\Downloads\3.Persi
012F1530	6F 77 6E 6C 6F 61 64 73 5C 33 2E 50 65 72 73 69	ownloads\3.Persi
012F1540	73 74 65 6E 63 65 5C 52 65 67 69 73 74 72 79 52	stence\RegistryR
012F1550	75 6E 4B 65 79 2E 65 78 65 00 00 00 00 00 00 00	unkey.exe.....
012F1560	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	
012F1570	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	

Chuyển đến API RegOpenKeyA.

7646CB05	8BFF	mov edi,edi	RegOpenKeyA
7646CB07	55	push ebp	
7646CB08	8BEC	mov ebp,esp	
7646CB0A	8B55 10	mov edx,dword ptr ss:[ebp+10]	[ebp+10]:&"ALLU
7646CB0D	85D2	test edx,edx	

Các tham số của hàm trong stack trước khi thực thi hàm:

```
Default (stdcall)
1: [esp+4] 80000001
2: [esp+8] 012FECFC "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run"
3: [esp+C] 0014FC6C
4: [esp+10] 77195F1C ntdll.77195F1C
5: [esp+14] 555C3A43

0014FC5C 012F103E return to registryrunkey.012F103E from ???
0014FC60 80000001
0014FC64 012FECFC "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\Run"
0014FC68 0014FC6C
0014FC6C 77195F1C ntdll.77195F1C
0014FC70 555C3A43
```

Đối với hàm này, tham số thứ nhất và thứ 2 đều là hằng số và xâu ký tự, còn tham số thứ 3 là 1 con trỏ đến 1 biến khác - biến đó nắm giữ handle đến Key được mở bởi hàm. Thực chất handle chỉ là 1 số nguyên (4 bytes). Ở đây, handle có giá trị là 64:

Address	Hex	ASCII
0014FC6C	64 00 00 00	d...C
0014FC7C	6E 67 5C 44	ng\Do
0014FC8C	65 72 73 69	ersis
0014FC9C	74 72 79 52	tryRu

Chuyển đến API RegSetKeyValueA. Các tham số trong stack trước khi thực thi hàm:

```
Default (stdcall)
1: [esp+4] 00000064
2: [esp+8] 00000000
3: [esp+C] 0095ED2C "RegistryRunKey"
4: [esp+10] 00000001
5: [esp+14] 0014F888 "C:\\Users\\trang\\Downloads\\3.Persistence\\RegistryRunKey.exe"

0014F898 0095105F return to registryrunkey.0095105F from ???
0014F89C 00000064
0014F8A0 00000000
0014F8A4 0095ED2C "RegistryRunKey"
0014F8A8 00000001
0014F8AC 0014F888 "C:\\Users\\trang\\Downloads\\3.Persistence\\RegistryRunKey.exe"
0014F8B0 00000104
0014F8B4 00000064
```

Đối với hàm này, cần theo dõi trong dump giá trị của tham số Filename. Giá trị trước khi thực thi hàm đã là path hiện tại của chương trình, do biến Filename đã lưu dữ liệu từ hàm GetModuleFileNameA trước đó.

Address	Hex	ASCII
0014F888	43 3A 5C 55	@:\Users\trang\D
0014F88C	6F 77 6E 6C	ownloads\3.Persi
0014F888	73 74 65 6E	stence\RegistryR
0014F8E8	75 6E 4B 65	unKey.exe..w8.R.
0014F8F8	E9 24 1B 77	ê\$.w..{s.....R.

API RegCloseKey chỉ đóng vai trò giải phóng tài nguyên nên có thể bỏ qua.

3. Kết luận

Tóm tắt hành vi:

1. Sử dụng API GetModuleFileNameA để trích xuất đường dẫn tuyệt đối của chính tệp thực thi đang hoạt động.
2. Mở khóa Registry Software\Microsoft\Windows\CurrentVersion\Run (thuộc phân vùng HKEY_CURRENT_USER) thông qua API RegOpenKeyA và lấy handle của khóa này.
3. Sử dụng API RegSetKeyValueA để tạo một giá trị mới bên trong khóa vừa mở, với dữ liệu được gán chính là đường dẫn tuyệt đối của tệp thực thi đã thu thập ở bước 1.
4. Gọi RegCloseKey để giải phóng handle.

Mẫu phân tích thực hiện hành vi thiết lập cơ chế Persistence thông qua [Registry](#). Bằng cách ghi đường dẫn trực tiếp vào khóa **Run** của hệ thống, chương trình đảm bảo sẽ được Windows tự động thực thi ngầm mỗi khi người dùng login.